Разработка программного модуля компьютерного зрения в PyBullet.

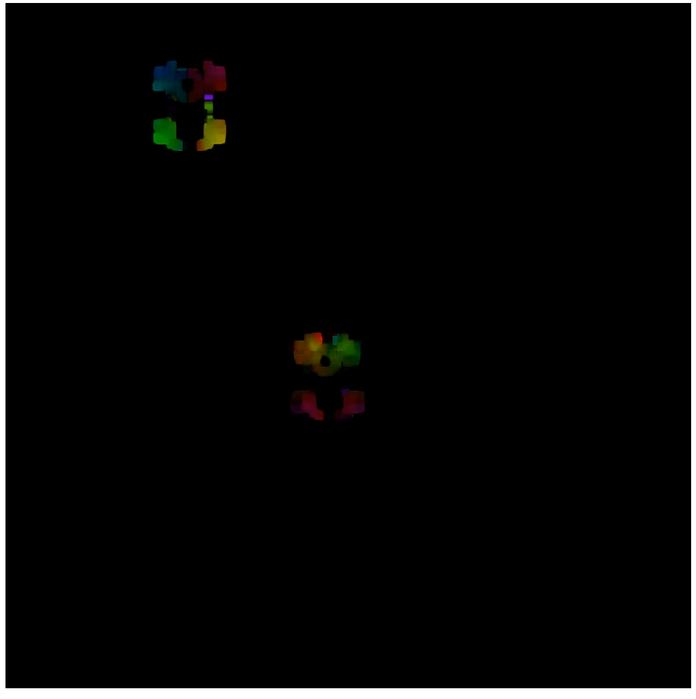
19.02.2026

Задача обнаружения движения

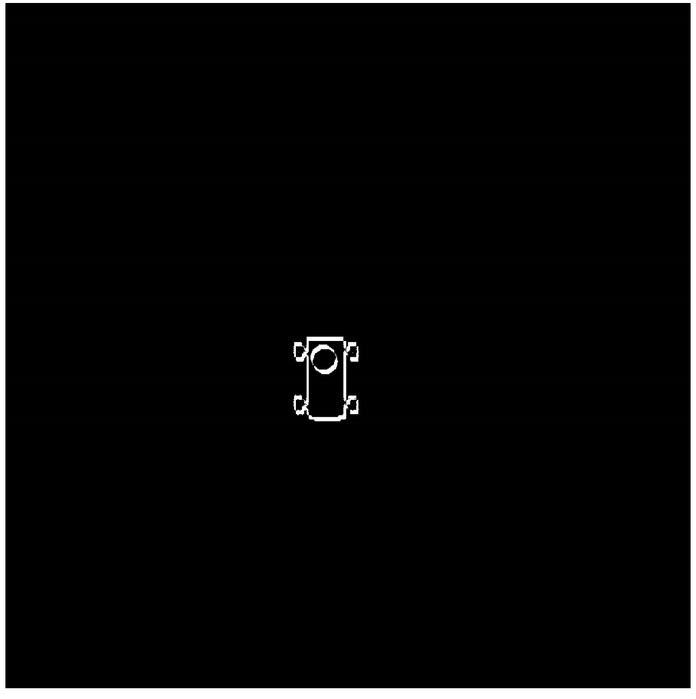
- неподвижная камера, постоянный фон, один движущийся объект
- то же с несколькими движущимися объектами
- движущаяся камера, постоянный фон, один или несколько движущихся объектов

Мы такое делать не будем, но картинка прикольная

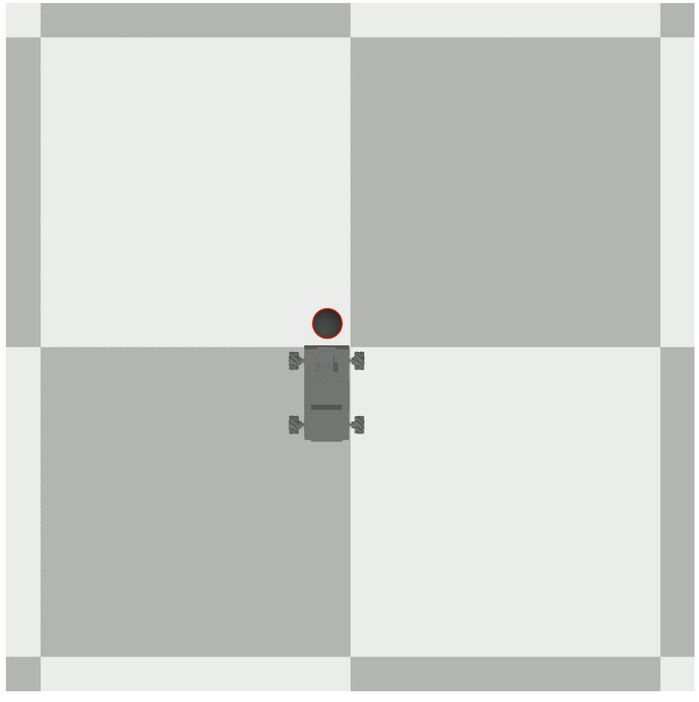
Задача обнаружения движения



Оптический поток (ОП) – изображение видимого движения, представляющее собой сдвиг каждой точки между двумя изображениями.



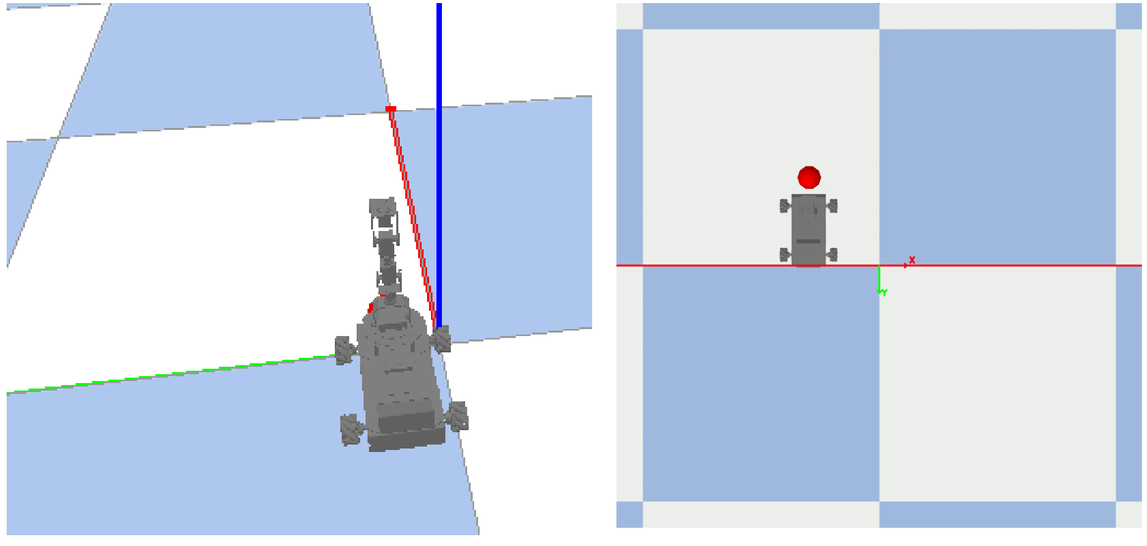
Вычитание изображений — это метод цифровой обработки, заключающийся в попиксельном вычитании значений интенсивности одного изображения из другого



Цветовой фильтр — это метод обработки изображений, позволяющий выделить определенные цвета (диапазон оттенков) и подавить остальные, часто используемый для поиска объектов.

Задача обнаружения движения

- Создание модели
- Создание маркера
- Создание камеры (вид сверху)
- Задание скорости робота
- Симуляция
- Расчет реальной скорости и реального времени симуляции
- Применение методов CV для определения скорости робота по видео (три разных способа)
- Сравнение результатов
- Отчет о границах применимости каждого из методов



Напоминание про оси

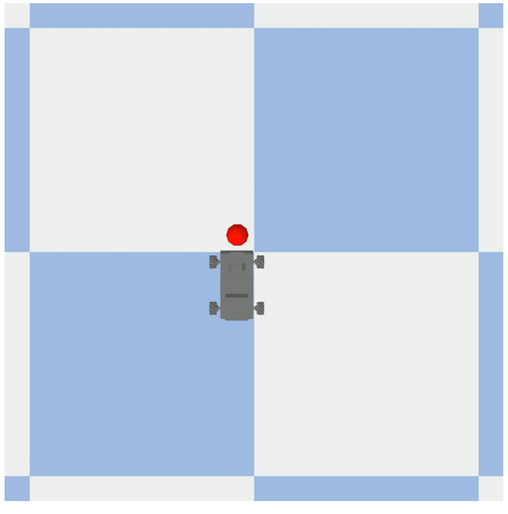
Задача обнаружения движения

- Создание модели
- Создание маркера
- Создание камеры (вид сверху)
- Задание скорости робота
- Симуляция
- Расчет реальной скорости и реального времени симуляции
- Применение методов CV для определения скорости робота по видео (три разных способа)
- Сравнение результатов
- Отчет о границах применимости каждого из методов

```
p.configureDebugVisualizer(lightPosition=[x, x, x])  задаем куда повесить свет
p.configureDebugVisualizer(p.COV_ENABLE_SHADOWS, 0) выключаем тени

cam_height = x  высота камеры
ortho = x    половина ширины/высоты ортографического “окна”

view_matrix = p.computeViewMatrix(
    cameraEyePosition=[x, x, cam_height],
    cameraTargetPosition=[x, x, x],
    cameraUpVector=[x, x, x]
)

projection_matrix = p.computeProjectionMatrix( #задаем ортографическую (параллельную) матрицу
проекцию камеры

# left/right/bottom/top – это границы окна просмотра в координатах камеры
    left=-ortho, right=ortho,
    bottom=-ortho, top=ortho,

    nearVal=900, farVal=1100
)


width, height, rgba, depth, seg = p.getCameraImage(
    width=1920,
    height=1920,
    viewMatrix=view_matrix,
    projectionMatrix=projection_matrix,
    renderer=p.ER_TINY_RENDERER   — встроенный CPU-рендерер
)
```

Задача обнаружения движения

- Создание модели
- Создание маркера
- Создание камеры (вид сверху)
- Задание скорости робота
- Симуляция
- Расчет реальной скорости и реального времени симуляции
- Применение методов CV для определения скорости робота по видео (три разных способа)
- Сравнение результатов
- Отчет о границах применимости каждого из методов



RUSSIAN SCIENCE IS FASCINATING!